Page Replacement

Dr. Jit Mukherjee

Motivation

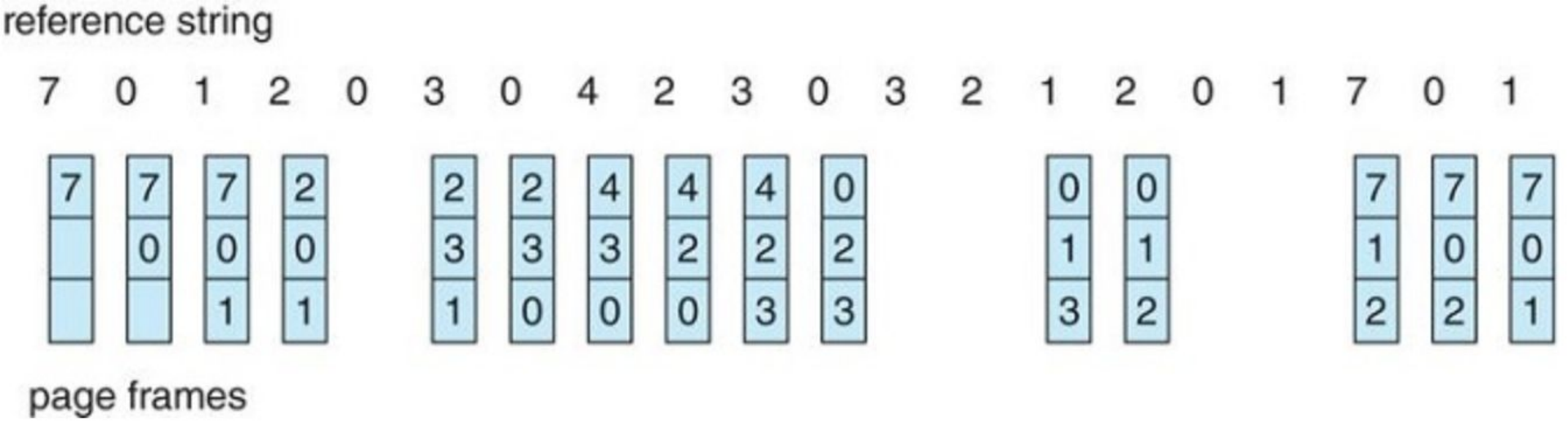
- We assumed each page fault at most once when first referenced
- If we increase degree of multiprogramming -> **over-allocating** memory
  - Six process, 10 pages, actually uses 5 pages, CPU 40 pages -> ten frame to spare ->If they suddenly need all of ten pages -> need of 60 from 40 pages
  - No free frame to spare
- System memory also has buffer for I/O -> How much memory to I/O or program -> Some system has predefined percentages -> Both can compete
- Terminate the a process -> User not aware of paging system -> not optimal
- Swap out a process -> freeing all the frames -> reducing Degree of Multiprogramming OR Page replacement
- Page-fault service -> Find the desired page -> Find a free frame, if there, use it
  - Page-replacement algo -> a victim frame -> Write the victim frame -> change page and frame table -> restart the process

Page Replacement

- If no frames are free, two page transfers are there -> one out and one in
  - Doubles the page fault service time, increase effective access time
- Modify bit or dirty bit - a bit associated with each page or frame
  - Whenever any word or byte is written into - set dirty bit. Page replacement -> we know this is modified -> must write the page to disk
  - If the page not modified -> modify bit not set. Need not write the memory pages to disk -. Already there
  - Read-Only pages -> May be discarded when desired.
- We evaluate an page-replacement algo by running it on a particular string of memory reference and computing the page faults - **Reference string**
- The number of frames available increases -> Page fault decreases.
  - 1,4,1,6,1,6,1,6,1,6,1
  - Three frames -> three faults. One frame -> 11 faults.

FIFO Page Replacement

- Associate each page with the time when it was brought into memory
  - Victim page -> Oldest Page
  - We can create a FIFO queue.
- Performance not optimal -> Page replaced may be a initialization module or heavily used variable which was initialized early



Belady's Anomaly

In the case of LRU and optimal page replacement algorithms,the number of page faults will be reduced if increase the number of frames.

However, Balady found that, In FIFO page replacement algorithm, the number of page faults will get increased with the increment in number of frames.

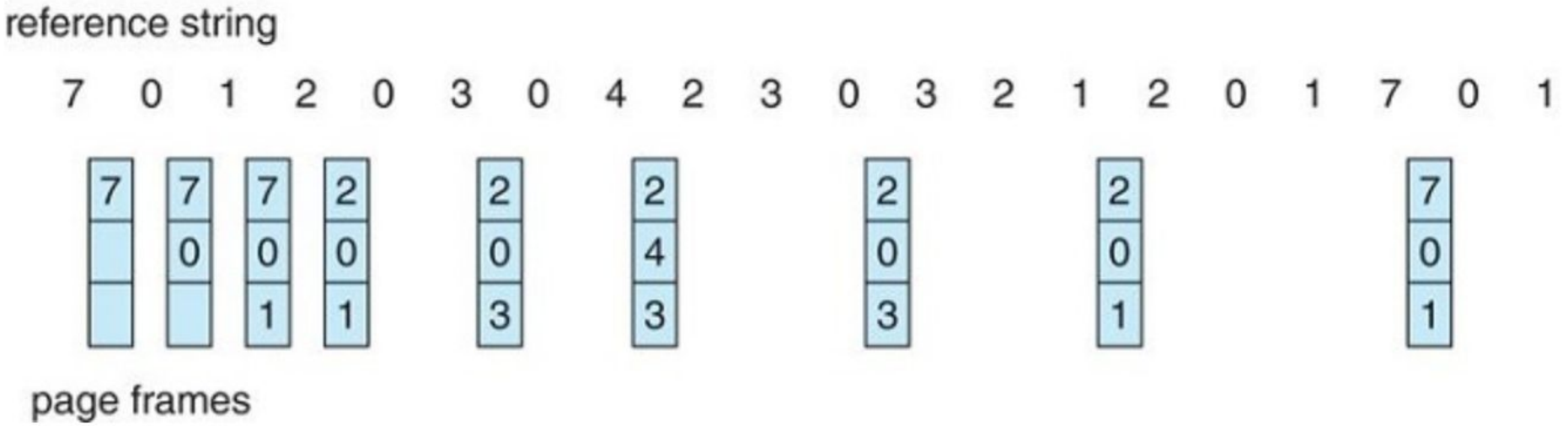
The reference String is given as 0 1 5 3 0 1 4 0 1 5 3 4. Number of Page Faults = 9 (3 frames), 10 (4 frames)

Request	0	1	5	3	0	1	4	0	1	5	3	4
Frame 3			5	5	5	1	1	1	1	1	3	3
Frame 2		1	1	1	0	0	0	0	0	5	5	5
Frame 1	0	0	0	3	3	3	4	4	4	4	4	4
Miss/Hit	Miss	Miss	Miss	Miss	Miss	Miss	Miss	Hit	Hit	Miss	Miss	Hit

Request	0	1	5	3	0	1	4	0	1	5	3	4
Frame 4				3	3	3	3	3	3	5	5	5
Frame 3			5	5	5	5	5	5	1	1	1	1
Frame 2		1	1	1	1	1	1	0	0	0	0	4
Frame 1	0	0	0	0	0	0	4	4	4	4	3	3
Miss/Hit	Miss	Miss	Miss	Miss	Ht	Ht	Miss	Miss	Miss	Miss	Miss	Miss

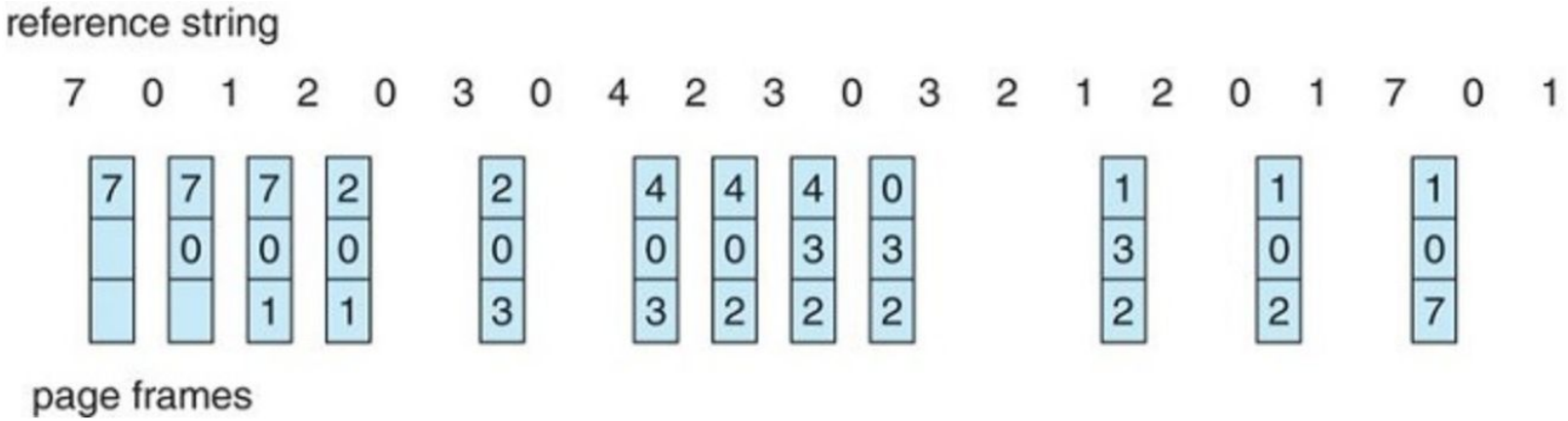
Optimal Page Replacement

- An optimal page replacement has the lowest page-fault rate and never suffers from belady's anomaly
- Replace the page that will not be used for the longest period of time
- It requires the future knowledge -> Similar to SJF CPU Scheduling



LRU Page Replacement

- An approximation of optimal. Looking backwards-> FIFO, Forward->Optimal. Replace the page that has not been longest period
- Associate each page with time of last use. **L**east **R**ecently **U**sed.
- Mostly used. Major problem is implementation
- S^R reverse of string S. Page fault rate for Opt and LRU -> S^R=S



LRU Page Replacement Contd.

- Implementation -
  - Counter - Associate each page table entry with time-of-use -> a logical clock or counter
    - A reference to a page-> Content of a clock to page-table entry
    - Replace the page with smallest time value
  - Stack - Keep a stack of page numbers
    - A reference to a page -> Remove from stack and put it in top
    - LRU is at the bottom of the stack
    - As middle element is accessed -> Doubly linked list
- LRU does not suffer from belady's anomaly. **Stack algorithms** can never exhibits belady's anomaly
  - A set of pages in memory for n frames is always a subset of the set of pages that would be in memory with n+1 frames
- Some systems provides sufficient hardware support -> LRU, Otherwise -> may be FIFO

LRU-Approximation Page Replacement

- **Reference bit** -> Whenever a page is referenced the bit is set.
  - Which pages have been used and which are not. We do not know the exact order
- **Additional Reference Bit Algorithm**
  - Record reference bit at regular intervals. Shifts the reference bit to a high order 8-bit byte.
  - 00000000 -> Never used. 11000100 > 01110100 -> more recently used. Take lowest number
- **Second Chance Algorithm**
  - FIFO with a twist. FIFO selects victim -> Check reference bit-> 0 replace -> if I, second chance
  - Select next FIFO page. Second chance page's reference bit is cleaned. Its arrival time is set to current -> It will be replaced last or all are given second chance
  - Circular queue
- **Enhanced Second Chance**
  - Reference bit and modify bit together -> (0,0) neither recently used or modified, best page to replace. (0,1) not recently used but modified - not quite good. (1,0) recently used but clean will be used again. (1,1)recently used and modified.
- **Counting Based Page Replacement**->No. of references made to each page
  - *Least Frequently Used* - Replace page with smallest count. Prob. Heavily used then discarded
  - *Most Frequently Used* - Page with the smallest count just bought in and yet to be used
- ***Page Buffering Algo*** - Keep a pool of free frames for quick start. Select a page, free it to the pool

Allocation of Frames

- A pool of free frames. Page swap and Page replacement
- Minimum number of frames -
  - MInimum number of frames per process -> Computer Architecture. Max -> Availability
  - Multiple Level of indirection.
- Allocation Algorithms -
  - Equal Allocation - m frames and n processes. Each process m/n frames. Other buffer pool
    - 93 frames, 5 process. 18 frames and 3 bufer
  - Proportional Allocation - a_i = s_i / S * m . s_i size of a process, m available memory
    - 62 frames, two process, 10 pages and 127 pages -> 10/137 * 62 = 4, 127/137 * 62 = 57
- Global and Local Allocation
  - Global - Allows a process to select from all set of frames, Local - Only those frames available to the process
  - High priority process is allowed to take from low priority processes
  - Local - number of frames for a process will remain same
  - Global - A process may mostly choose from other processes space